

# Исключения

## 1. Основная терминология

**Исключение** – это некоторый нестандартный случай (исключительная ситуация), который не может быть обработан.

**Исключение в C++** – это объект некоторого типа, который генерируется в исключительной ситуации.

С исключениями связаны 3 оператора:

- `throw` («бросать» / «кидать»)
- `try` («пробовать» / «пытаться»)
- `catch` («ловить» / «поймать»)

Оператор `throw`, как следует из названия, позволяет «бросать» (генерировать) исключение. За этим оператором должен быть указан **объект**, который будет брошен. Это может быть как объект стандартного типа, так и объект пользовательского типа. Объект может определяться сразу после оператора `throw`, также можно бросать ранее определённые переменные.

Внутри оператора `try` помещается блок кода, в котором может генерироваться исключение, *которое мы хотим поймать*.

В блоке оператора `catch` помещается код, который должен выполняться, если исключение произошло и было «поймано». В качестве параметра `catch` передаётся тип исключения, которое должно быть поймано, или многоточие – если должно быть поймано исключение любого типа. Параметры блока `catch` аналогичны параметрам функций – их так же можно передавать по значению и по ссылке, они тоже могут быть константными. После выполнения кода в блоке `catch` выполняется код, следующий за этим блоком.

## 2. Основные механизмы использования

Простой пример – деление на ноль – приведён в примере 1. В блоке `try` помещён код, генерирующий исключение. Если делитель равен нулю, то «бросаем» -1 (строка 13). Т.к. в блоке `catch` стоит многоточие и ловятся все исключения, программа ловит -1 и выводит сообщение о делении на 0 (строка 19). Данный пример носит иллюстративный характер. Какие исключения в каких ситуациях бросать правильно, будет рассмотрено позднее.

**У блока `try` должен быть хотя бы один `catch`!** Блоков `catch` может быть несколько, они должны отличаться типами параметров (пример 2). Если бросаемое значение не будет использовано и важен только тип исключения, может быть указан только тип значения (пример 3), если значение будет использовано, то необходимо указать название для переменной, куда будет помещено значение (пример 4).

Поиск подходящего блока `catch` осуществляется последовательно, так что в **случае использования универсального `catch` (с многоточием), он должен быть размещён самым последним.**

### Пример 1. Деление на 0

```
1  #include <iostream>
2
3  int main()
4  {
5      float a = 0, b = 0;
6      std::cout << "Enter a: ";
7      std::cin >> a;
8      std::cout << "Enter b: ";
9      std::cin >> b;
10     try
11     {
12         if (b == 0)
13             throw -1;
14         else
15             std::cout << "Result: " << a/b;
16     }
17     catch(...)
18     {
19         std::cout << "Division by zero\n";
20     }
21     return 0;
22 }
```

### Пример 2. Пример размещения блоков catch при обработке исключений типа char и int

```
1  catch(char) // если исключение типа char, выполнится этот блок
2  {
3      // инструкции
4  }
5  catch(int) // если исключение типа int, выполнится этот блок
6  {
7      // инструкции
8  }
9  catch(...) // если исключение другого типа, выполнится этот блок
10 {
11     // инструкции
12 }
```

### Пример 3. Деление на 0 с «типизированным» catch без имени переменной

```
1  #include <iostream>
2
3  int main()
4  {
5      float a = 0, b = 0;
6      std::cout << "Enter a: ";
7      std::cin >> a;
8      std::cout << "Enter b: ";
9      std::cin >> b;
10     try
11     {
12         if (b == 0)
13             throw -1; // тип исключения - int
14         else
15             std::cout << "Result: " << a/b;
16     }
17     catch(int)
18     {
19         std::cout << "Division by zero\n";
20     }
21     return 0;
22 }
```

#### Пример 4. Деление на 0 с «типизированным» catch с именем переменной

```
1  #include <iostream>
2
3  int main()
4  {
5      float a = 0, b = 0;
6      std::cout << "Enter a: ";
7      std::cin >> a;
8      std::cout << "Enter b: ";
9      std::cin >> b;
10     try
11     {
12         if (b == 0)
13             throw -1; // тип исключения - int
14         else
15             std::cout << "Result: " << a/b;
16     }
17     catch(int e) // в e будет лежать -1
18     {
19         std::cout << "Error code " << e << ": Division by zero\n";
20     }
21     return 0;
22 }
```

Кидать исключение не обязательно непосредственно в блоке `try`. Код, бросающий исключение, может находиться в функции, которая вызывается внутри блока `try`. Уровень вложенности может быть любым: когда генерируется исключение и блок `try` не находится внутри функции, где сгенерировано исключение, блок ищется в «вышестоящей» функции, если не найден там – ещё в более «вышестоящей», и т.д., пока не найдётся блок `try` или не закончатся в функции. Если поиск дошёл до `main`, и там тоже нет `try`, исключение считается необработанным, программа падает (завершается аварийно). Пример с бросанием исключения в функции – в примере 5.

#### Пример 5. Деление на 0 с вынесением в отдельную функцию

```
1  #include <iostream>
2
3  double division(double const& a, double const& b)
4  {
5      if (b == 0)
6          throw -1;
7      return a / b;
8  }
9
10 int main()
11 {
12     float a = 0, b = 0;
13     std::cout << "Enter a: ";
14     std::cin >> a;
15     std::cout << "Enter b: ";
16     std::cin >> b;
17     try
18     {
19         std::cout << "Result: " << division(a, b);
20     }
21     catch(...)
22     {
23         std::cout << "Division by zero\n";
24     }
25     return 0;
26 }
```

Поскольку после бросания исключения выполнение программы переходит к соответствующему блоку `catch`, инструкции после `throw`, не будут исполнены.

### 3. Исключения и ООП

Разумеется, мы не просто так говорим об исключениях во втором семестре ☺

Несмотря на то, что большинство из вас не кидало исключения, едва ли кто-то с ними не сталкивался, когда его программа падала.

Стандартные библиотеки генерируют [множество исключений](#), которые являются наследниками класса `std::exception`. В этом классе есть *виртуальный* метод с говорящим названием `what`, с помощью которого можно получить строку-пояснение (`const char*`), которая может быть задана реализацией `what` либо может быть взята из некоторого поля класса.

Наследники `std::exception` разделены по ситуациям, в которых они возникают. Например, исключения типа `std::logic_error` и его наследники возникают в случаях, когда возникла ошибка, связанная с неправильной логикой работы программы. Так, в случае неправильного аргумента вызывается `std::invalid_argument`. Именно это исключение было бы более подходящим в примере с делением на 0.

#### Пример 6. Деление на 0 с использованием стандартного исключения

```
1  #include <iostream>
2  #include <stdexcept>
3
4  double division(double const& a, double const& b)
5  {
6      if (b == 0)
7          throw std::invalid_argument("Division by zero");
8      return a / b;
9  }
10
11 int main()
12 {
13     float a = 0, b = 0;
14     std::cout << "Enter a: ";
15     std::cin >> a;
16     std::cout << "Enter b: ";
17     std::cin >> b;
18     try
19     {
20         std::cout << "Result:" << division(a, b);
21     }
22     catch (std::exception& e)
23     {
24         std::cout << e.what();
25     }
26     return 0;
27 }
```

В примере 6 приведена демонстрация генерации исключения путём вызова конструктора с параметром для `std::invalid_argument` (строка 7). В качестве параметра передаётся строка сообщения.

```
Enter a: 5
Enter b: 0
Result: Division by zero
```

Стоит обратить внимание, что «ловля» исключения происходит через ссылку на `std::exception` (строка 22). Этот механизм работает только в случае, когда генерируется исключение-наследник, а «ловится» исключение базового класса. В обратную сторону (генерируется «родитель», ловится «наследник») не работает. Также обращаю внимание, что для того, чтобы в этом случае гарантированно было выведено верное сообщение об ошибке, должен сработать динамический полиморфизм, для чего необходимо «ловить» исключение по ссылке. В противном случае результат будет зависеть от реализации используемой вами версии стандартных библиотек (если там `what` для `std::exception` выводит строковую константу, а не значение поля, то будет выведена константа, а переданное в конструктор сообщение будет потеряно).

В случае возможной генерации исключений-наследников и исключений-родителей и необходимости их по-разному обрабатывать, обработчик для исключения-наследника нужно располагать выше обработчика исключения-родителя.

При необходимости можно определить свои исключения. Для унификации работы следует определять новые исключения как наследников `std::exception` или его наследников. У нового класса-исключения переопределяется метод `what` и по необходимости добавляется поле, в которое можно с помощью конструктора с параметром записать сообщение.

#### 4. Опасности исключений

Вы могли заметить, что при генерации и обработке исключений нарушается «правильный» порядок действий, если в блоке `try` исполняются не все инструкции. Порой это весьма фатально: например, при работе с файлами и динамически выделенной памятью.

В примере 7 приведена как раз такая ситуация: со строки 4 мы переходим на строку 7 и далее на строку 9. Таким образом, происходит утечка памяти.

##### Пример 7. Утечка памяти

```
1  try
2  {
3      int* a = new int[10];
4      someFunction(a); // Где-то здесь генерируется исключение
5      delete[] a;
6  }
7  catch(...)
8  {
9      std::cout << "Error\n";
10 }
```

У ситуации из примера 7 есть два очевидных решения: чистить память и в `catch` (пример 8) и вынести очищение памяти после `catch` (пример 9). Стоит упомянуть, что в этих случаях объявление переменной `a` должно быть вынесено из блока `try`, иначе она не будет доступна.

#### Пример 8. Дублирование очищения памяти

```
1  int* a = nullptr;
2  try
3  {
4      a = new int[10];
5      someFunction(a); // Где-то здесь генерируется исключение
6      delete[] a;
7  }
8  catch(...)
9  {
10     std::cout << "Error\n";
11     delete[] a;
12 }
```

Первый вариант плох тем, что происходит дублирование (и про него нужно не забыть), второй – тем, что память не обязательно будет выделена (перед выделением памяти могут быть другие инструкции, бросающие исключения). Конечно, данном примере это сделать очень сложно, в нём мало строк. Но даже в более-менее нормальной ЛР строк кода гораздо больше, и `delete` частенько забывается без всяких исключений.

#### Пример 9. Вынесение очищения памяти

```
1  int* a = nullptr;
2  try
3  {
4      a = new int[10];
5      someFunction(a); // Где-то здесь генерируется исключение
6  }
7  catch(...)
8  {
9      std::cout << "Error\n";
10 }
11 delete[] a;
```

Идеальным вариантом в таких ситуациях является очищение памяти при выходе переменной из области видимости. Надеюсь, теперь вы понимаете, почему я так призываю очищать память в деструкторе ваших классов ☺

## 5. Идиома (концепция) RAII

**RAII** (*Resource Acquisition Is Initialization*, получение ресурса есть инициализация) – подход, в соответствии с которым выделение памяти (получение ресурса) неразрывно связано с его инициализацией, а освобождение – с уничтожением самого объекта. Такой подход позволяет избежать в т.ч. проблем в случае досрочного завершения выполнения функций.

## 6. Вложенные блоки обработки исключений

Блоки обработки исключений поддерживают вложенность. Связка `try-catch` может быть вложена в блок `try`, а внутри `catch` могут генерироваться новые исключения. Вложенность может быть выражена явно, а также может проявляться в вызове внутри блоков функций, содержащих операторы работы с исключениями.

Если исключение не должно быть обработано (до конца) в блоке `catch` на том же уровне, где оно было сгенерировано, его можно передать в вышестоящий блок вызовом `throw` без указания объекта. Этот способ является предпочтительным, т.к. при передаче словленной переменной или

генерации нового исключения тратится дополнительная память (создаётся копия объекта), а также возможны проблемы с типом, когда исключение было словлено блоком `catch` для родственного типа.

## 7. Спецификация исключений

Есть механизм спецификации исключений, который в большей части редко использовался на практике. Этот механизм позволяет указать, какие исключения может генерировать функция (и может ли вообще). Спецификация производится путём указания `throw()` в конце объявления функции.

Есть 3 варианта спецификации (параметров `throw`):

- исключения конкретного типа – параметром указывается тип или несколько типов;
- исключения любого типа – параметром указывается многоточие;
- никакие исключения – скобки остаются пустыми (**единственный используемый случай**).

Со стандарта C++11 для запрета генерации исключений вместо `throw()` рекомендуется использовать спецификатор `noexcept`. В новых стандартах варианты со `throw()` уже исключены.

Пример 10. Функция деления со спецификатором исключения (старый стандарт)

```
1 double myFunction() throw()
2 {
3     // код
4 }
```

Пример 11. Функция деления со спецификатором исключения (новый стандарт)

```
1 double myFunction() noexcept
2 {
3     // код
4 }
```

Стоит отметить, что спецификаторы запрещают функции бросать исключения (все или все, кроме разрешённых типов) **наружу**. Функции с запрещающими спецификаторами могут генерировать исключения и обрабатывать их внутри себя, не передавая наружу.

**+1 best practices:** используйте `noexcept` для конструкторов копирования и перемещения, операторов присваивания и перемещения, деструкторов. Для корректной работы с ресурсами эти методы должны выполняться полностью и не должны прерываться.

## 8. Функциональный блок `try`

Для того, чтобы применять блок `try` ко всему телу функции, есть особый – т.н. «функциональный» вариант блока `try`. Он позволяет «прицепить» обработку исключения, генерируемого функцией, к самой функции. В примере 12 показано использование функционального `try` применительно к ситуации из примера 5.

В отличие от примера 5, где строка с результатом не будет выведена на экран, в этом случае она выведется на экран, т.к. она находится вне блока `try` и выполняется в любом случае.

### Пример 12. Функциональный блок try

```
1  #include <iostream>
2
3  double division(double const& a, double const& b) try
4  {
5      if (b == 0)
6          throw -1;
7      else
8          return a / b;
9  }
10 catch(...)
11 {
12     std::cout << "Division by zero\n";
13 }
14
15 int main()
16 {
17     float a = 0, b = 0;
18     std::cout << "Enter a: ";
19     std::cin >> a;
20     std::cout << "Enter b: ";
21     std::cin >> b;
22     std::cout << "Result: " << division(a, b);
23     return 0;
24 }
```